The Journal of Engineering

The 3rd Asian Conference on Artificial Intelligence Technology (ACAIT 2019)

IET Journals
The Institution of Engineering and Technology

Research on integrated computer game algorithm for dots and boxes

eISSN 2051-3305
Received on 14th October 2019
Accepted on 19th November 2019
E-First on 27th July 2020
doi: 10.1049/joe.2019.1185
www.ietdl.org

Shuqin Li[1,2] ✉, Yipeng Zhang[1,2], Meng Ding[1,2], Pengcheng Dai[1,2]

[1]*Computer Academy, Beijing Information Science & Technology University, Beijing, People's Republic of China*
[2]*Perception And Computation Intelligence Joint Lab, No.35, north fourth ring road, chaoyang district, Beijing, People's Republic of China*
✉ *E-mail: lishuqin_de@126.com*

Abstract: Situation assessment and search are two key problems in computer game research. In general, as the game progresses, the difficulty of evaluating the situation of the game is significantly reduced, and the accuracy of the evaluation is significantly increased. Based on the famous chess game, this article proposes and implements a new scheme that combines the Monte Carlo tree search algorithm, the Alpha-Beta algorithm and the model based on the deep convolution neural network (CNN) to solve the computer game problem. This article first proposes a deep convolutional neural network model based on dots and boxes, including deep value network and deep strategy network, focusing on situation assessment and strategy recommendation, respectively. Then, using the Monte Carlo Tree Search (MCTS) algorithm as a framework, deep value network integrated MCTS algorithm and deep strategy network integrated MCTS algorithm are proposed. In both integrated models, Alpha-Beta complete search is used to truncate the Monte Carlo simulation process and improve simulation efficiency. Through competition with human players, the results show that the two integrated algorithm game systems have reached much higher intelligence level than ordinary humans in solving the problem of dots and boxes.

1 Introduction

Computer game is one of the most challenging application areas of artificial intelligence. The process of playing chess is that the two sides alternately give the playing method, which makes the chess game develop in the favourable direction of the player until the final victory. Therefore, the core of the computer chess game is how to give the game, that is, using the method to deduce the situation, choose the current game from the favourable situation, the essence of which is to construct the search tree and evaluate the search tree to determine how to, that is, determine the direction of the search tree. It is not difficult to see that evaluation and search will become an important part of the game software, so the main research of computer game is also focused on these two key issues.

Initially, the maximum-minimum search based on depth-first search is used as a general method for game state tree search in computer game systems. Then, the Alpha-Beta pruning method [1] based on maximum minima search is proposed and obtained. In order to avoid the Alpha-Beta search process, especially its dependence on game experience, the Monte Carlo Tree Search (MCTS) [2] algorithm emerged. It simulates the objective situation winning rate through a large number of random games, and then solves the game problem. It has good versatility and controllability. However, the MCTS algorithm suffers from its natural statistical properties. It needs to seek a balance between objective statistics and preference mining in the search process. This makes a large number of random simulations of MCTS difficult to avoid including a large number of redundant calculations. After the DeepMind team introduced deep learning technology into computer games [3–5], the method of integrated deep learning [6] has received extensive attention and considerable development in the field of computer games. A well-trained convolutional neural network (CNN) [7] model can immediately give an assessment of a game situation and has a strong generalisation capability, while the MCTS performs on the time overhead and evaluation accuracy. In the middle, if CNN is integrated with the MCTS algorithm, the overall evaluation accuracy of the algorithm can usually be improved at the expense of a small amount of time. The success of the AlphaGo [8] system also confirms this.

Based on the 5×5 dots and boxes problem [9], we propose and implement a new solution to solve the computer game problem by combining MCTS algorithm, Alpha-Beta algorithm and deep CNN model. It is intended to be an exploration for the next step in the development of computer game general platform work.

2 CNN model for dots and boxes

2.1 Introduction to the dots and boxes rules

The dots and boxes game is a well-known two-player board game that has been incorporated into the International Computer Olympiad Competition and the China Computer Gaming Competition for many years. It has many names, such as dot-to-square checkerboard, encircle chess and so on. The rule of dots and boxes is as follows: In a rectangular array of a certain size and evenly distributed, two players alternately draw horizontal or vertical straight lines in their own turn, connecting two adjacent points. If the player draws a line, any unit square enclosed by four points is closed, that is, the line is drawn around the line. The line drawing player captures the frame and scores one point. The scoring player must continue to move one round. When the line cannot be drawn anymore, that is, when all the lattices in the lattice are occupied, the game ends and the player occupying the most lattice wins. Fig. 1 shows a 3×3 dots and boxes game between Arthur and Beth, with Arthur as the ancestor [10].

2.2 Terms relating to dots and boxes

In the mid to late stage of the dots and boxes game, there are usually several chain structures in the chess game.

Definition 1: Chain

The chain in the dots and boxes refers to the 'channel' that is formed by the head and tail.

In this paper, we classify the chains in 5×5 dots and boxes, which are short chain, medium chain and long chain, respectively. Among them, the short chain is a chain containing only one cell; the middle chain is a 'path' containing two cells or containing four squares of 'rings', long chains are longer or intermediate chains than medium chains. The specific classification criteria are shown in Fig. 2.

If there is a player in the chain structure who creates an opportunity to score points by adding an edge, it will 'activate' the

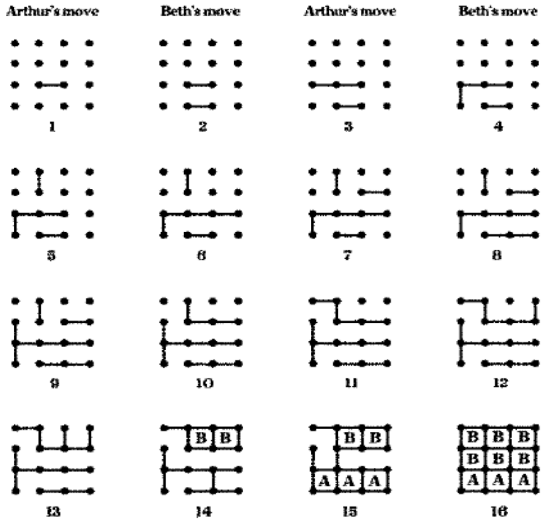

Fig. 1 *Dots and boxes game between Arthur and Beth*

chain, giving the opponent the chance to score consecutively until the entire chain is occupied.

Definition 2: Change hands

In the case that the player can continuously occupy a number of grids and score points in his own turn, by giving up part of the grid and transferring the right of action to the opponent in advance, the player is said to have changed hands.

Changing hands is an important strategy in the dots and boxes game, based on a rule in the dots and boxes: 'The scoring player must continue to move one turn'. When the situation develops into a safety move that does not make opponents score less, at the cost of as few points as possible, the opponent is forced to continue the action and lose more scores after the exchange round, making the total score as large as possible. This is the point. The basic principle of hand-off strategy in chess. The typical hand change type is shown in Fig. 3. The medium chain just contains a chance to change hands.

2.3 Dots and boxes game expression and CNN structure

In all of the dots and boxes computer game implementations, the status of the game needs to be formatted to describe, that is, a specific data structure is required to completely describe various situation characteristics, such as the present status of the edge, the status of the grid, and the degree of the grid. Existing marginal number, both scores and current round off attribution. In this paper, the 11×11 two-dimensional matrix abstractly and formatively represents the edge, grid and lattice belonging information in the various situations of 5×5 dots and boxes, as shown in the top two figures in Fig. 4. Among them, the block marked with R represents the grid that has been occupied by the red player; the marked B block represents the grid that has been occupied by the blue player; each representative grid has one representative edge in four directions: up, down, left and right. Blocks represent the states of the four sides that are adjacent to the grid and the state of the specific grid in the checker board. The tag 1 indicates the edge that has been occupied; the tag 0 indicates the unoccupied edge. The grey region is useless point. In addition, in order to allow the CNN model to learn deeper logic, it is necessary to artificially refine some of the hidden information in the game of dots and boxes, such as typical local patterns in the situation, including short chain, medium chain, long chain and changing hands. We refine 17 game state features. The attribute properties of all attribute designs are shown in Table 1.

This paper proposes a 17-level stacked binary 11×11 matrix to express the expression of a certain dots and boxes, in which each layer separately describes the status and implied features of various positions. That is, the input of the CNN is $11 \times 11 \times 17$ imaged data, as shown in Fig. 4.

length	rings or path	sort chain	example
1	path	short	
2	path	medium	
≥3	path	long	
4	rings	medium	
5	rings	long	

Fig. 2 *Classification criteria of chains*

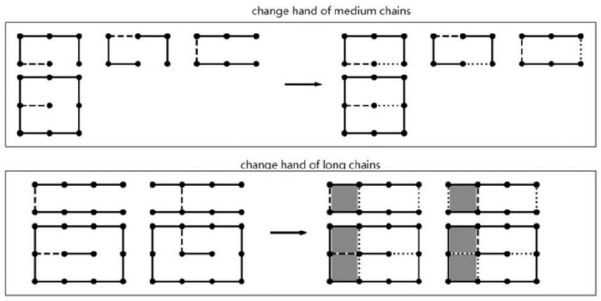

Fig. 3 *Typical hand change diagram*

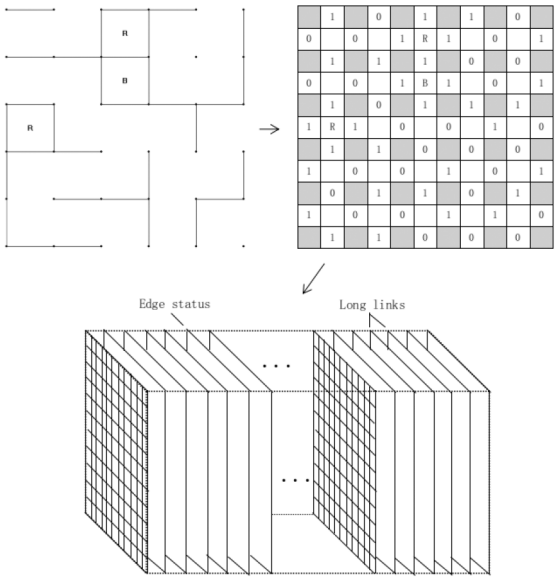

Fig. 4 *Dots and boxes situation information representation and CNN network structure*

3 Deep learning algorithm for dots and boxes

How to give an objective score for each game situation and draw the next best practice is to design one of the difficulties in the computer game system. In this paper, the deep neural network model is used as a regression to evaluate the quality of both sides of players in real-time, and to evaluate the current situation of all

Table 1 Game state features

Feature	The number of matrix layers	Description
all 1	1	contrast use all 1 matrix
all 0	1	contrast use all 0 matrix
side area	1	edge corresponds to the legal matrix position
grid area	1	grid corresponding to the legal matrix position
colour	1	whether the current player is blue
side state	1	whether the edge is occupied
grid state	3	return to the red/ return to blue/ free
short chain	2	short chain position and quantity
medium chain	2	medium chain position and quantity
long chain	2	long chain position and quantity
score grid	1	the grid that can instantly occupy the score
change hands grid	1	A grid that can be used as a trade-off

possible branches to select the best moves to solve the problem of dots and boxes computer games.

3.1 Design and implementation of a deep value network model

The input of the deep value network model in this paper adopts a dots and boxes situation information, namely a $17 \times 11 \times 11$ three-dimensional matrix. The model output is a single value, which is the evaluation of the game situation. After the data sample is input, it will go through seven layers of convolution layers. The number of convolution kernels is 192 in each layer, the convolution kernel size is 3×3, and the convolution kernel horizontal and vertical movement steps are all 1, after the convolution operation. The output matrix is complemented by 0, the input data size remains unchanged at 11×11, and the activation function after the convolution operation selects the Exponential Linear Unit (ELU) [11]. The down-sampling operation is not performed after the convolution operation. After seven layers of convolutional data, the data enters into two fully connected layers. The number of neurons in each fully connected layer is 192, and the non-linear activation function still uses the ELU function. Finally, the data enters the output layer, which contains a neuron structure. The data entering the output layer does not need to pass through a non-linear activation function. The Adam algorithm [12] is more stable than the stochastic gradient descent algorithm. It continuously iteratively optimises the connection weights of the convolution kernel and the full connect layer of all convolutional layers in the network according to the error between the network output and the expected value. The output of the model is normalised by the Sigmoid function and falls in the [0,1] interval. The deep value network structure is shown in Fig. 5.

When supervised deep learning is used to train the depth value network, the error function is set as the mean-squared deviation of the model output corresponding to the sample value in each mini batch, $V(s,a,\theta)$, and the sample calibration z. 1 shows:

$$L_V = E[(z - V(s,a,\theta))^2] \quad (1)$$

Among them, z represents the ground truth value of the position value, s in $V(s, a, \theta)$ represents a specific point grid chess game situation, a represents a movement that makes the position into position s, and θ represents a neural network model of the weight parameters.

Depth value network optimisation algorithm uses Adam algorithm, set the basic learning rate of Adam optimisation algorithm is 0.001, α parameter is 0.9, β parameter is 0.99, training mini batch size is 50, that is, each training iteration, enter 50 models into the model. The situation samples are calculated in parallel by the multiple kernel structures of the GPU device and deduced at the same time. In the training process, 100 mini batches are saved for each training model parameter, and only the latest 100 model parameters archive versions are kept during the training.

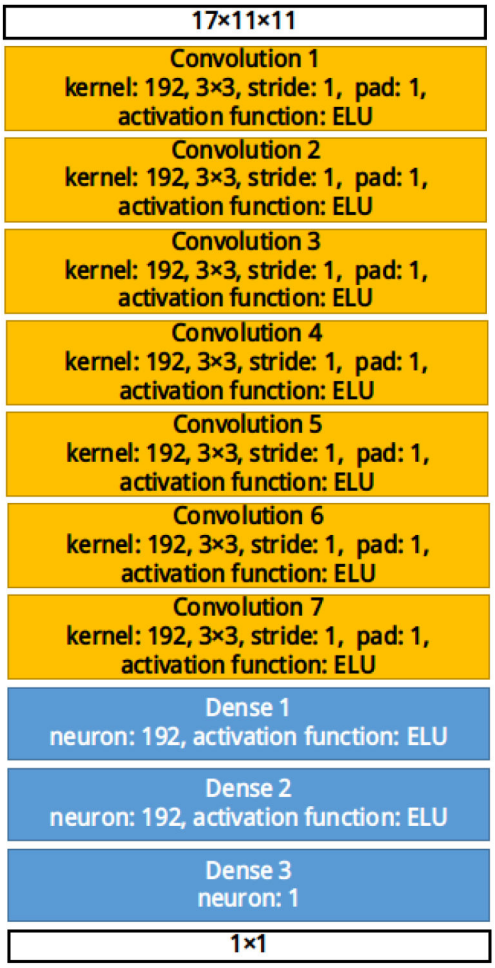

Fig. 5 *Depth value network*

3.2 Design and implementation of deep strategy network model

Compared to the use of deep value networks, it is more efficient to directly select models to choose the best approach based on situation information. This paper designs an in-depth strategy network model that is similar to AlphaGo Lee's strategy network and can directly output the best approach.

In this paper, the depth strategy network model input and depth value network adopt the same dots and boxes situation information, namely $17 \times 11 \times 11$ three-dimensional matrix. The model output is a one-dimensional vector of length 60 that describes the degree of confidence that the particular approach plays on the 60 possible action areas on the board, i.e. the degree of propensity to 60 possible decisions. After the data sample is input, it passes through five layers of convolution layers. The number of convolution kernels in each layer is 64, the convolution kernel size is 3×3, and the convolution kernel length is 1 in both longitudinal and longitudinal movement steps. After the convolution operation, the output matrix is complemented with 0, the data size at the input is maintained, the activation function after the convolution operation selects the ELU, and the down-sampling operation is not performed after the convolution operation. After five layers of convolutions, the data enters a fully connected layer of one layer. The number of neurons is 256, and the activation function still uses ELU. The output stream of the full connected layer consists of a dropout layer for random screening, the retention rate is set to 0.8, which weakens the local correlation in the data stream matrix and enhances the overall generalisation ability of the neural network model. The final data enters the output layer, which contains 60

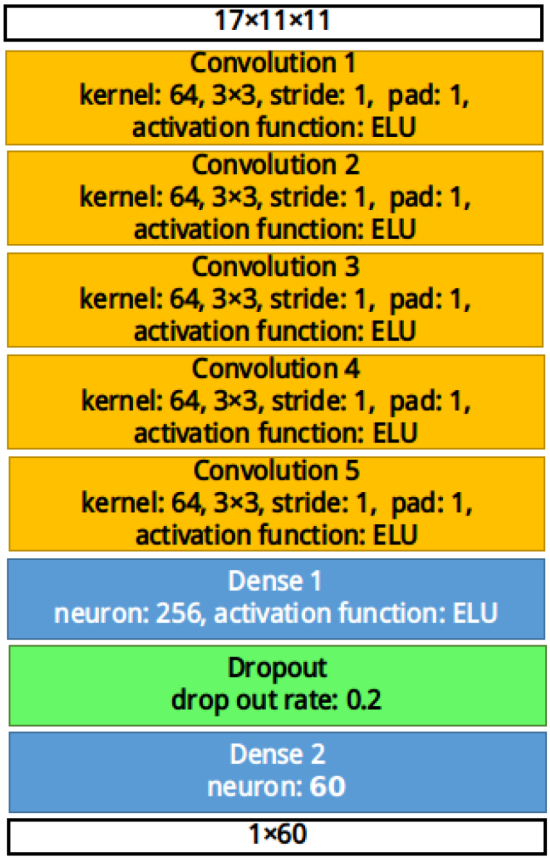

Fig. 6 *Depth strategy network*

neuron structures. The data entering the output layer does not need to pass through the non-linear activation function. The Adam algorithm, which is more stable than the stochastic gradient descent algorithm, is used to iteratively optimise the concatenated kernel and concatenated neuron connection weight values of all convolutional layers in the network according to the error between the network output and the expected value. Finally, the output of the model is normalised by the soft max function so that each output value, that is, the confidence of the movement in a particular possible action area, falls in the [0, 1] interval. The network structure of the in-depth strategy is shown in Fig. 6.

When supervised deep learning is used to train the depth strategy network, the error function is set to the cross-entropy of the model output strategy score $P(s,a,\theta)$ and the sample calibration π corresponding to each mini batch sample, that is:

$$L_P = -E[\pi\log(P(s,a,\theta)) + (1-\pi)\log(1-P(s,a,\theta))] \quad (2)$$

The optimisation algorithm of the in-depth strategy network uses Adam algorithm, sets the basic learning rate of the Adam optimisation algorithm to 0.001, the α parameter to 0.9, the β parameter to 0.99, and the mini batch size for training is 32. That is, in each training iteration, the input to the model is 32. Samples of positions are calculated in parallel by multiple kernel structures of the GPU device and deduced at the same time. In the training process, 100 mini batches are saved for each training model parameter, and only the latest 100 model parameter archive versions are kept during the training.

4 Integrated deep learning algorithm

End-to-end use of a deep learning model for computer gaming, that is, only the deep learning model provides intelligent decision-making for different situations, which may lead to instability of the intelligence level of the computer game system or unbalanced power in the face of different game situations. The reason is that it is difficult to ensure that the deep learning model is fully trained in all game situations. However, integrating the deep learning model with the MCTS algorithm can make up for the deficiency of the deep learning model and enhance the overall intelligence level of the computer game system.

4.1 Depth value network integrated MCTS algorithm

The MCTS algorithm is an online search algorithm that builds a game tree probability model through a large number of random simulations. In order to improve the search efficiency, algorithms such as upper confidence bound apply to tree (UCT) [13] apply a specific balancing strategy in the MCTS algorithm, such as the typical [14] upper confidence bound (UCB) formula, which tends to have a tendency at each state node. Select the branch node. The balancing strategy in the MCTS algorithm is designed to make the choice of both branches take into account both tendencies: uniform exploration and evaluation of all legitimate branches, focusing on trying branches with higher probability of victory. The UCB formula is shown in (3).

$$v_i = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N}{n_i}} \quad (3)$$

Among them, w_i is the number of times the current mobile player wins in all simulations under the ith branch under a specific situation, n_i is the total number of simulations under the ith branch under a specific situation, N is the total number of simulations under a specific situation, c is the exploration tendency parameter, usually set to $\sqrt{2}$.

In this paper, the depth value network is integrated into the MCTS algorithm, that is, the depth value network evaluates the corresponding situation of leaf nodes in each first visited game state. After the evaluation, the current simulation continues until the results of the simulation match are obtained. The evaluation of each dots and boxes will consist of the simulation result and the output of the depth value network. That is to say, w_i/n_i in formula (4) is replaced by a weighted sum. Formula (4) is the equilibrium strategy formula used in this paper.

$$v_i = \lambda\frac{w'_i}{n'_i} + (1-\lambda)\frac{w_i}{n_i} + c\sqrt{\frac{\ln N}{n_i}} \quad (4)$$

where λ is the confidence value of the depth value network and the range is [0, 1]. w'_i is the cumulative sum of the deep value network evaluation values of all simulations under the ith branch in a specific situation, and n'_i is the total number of evaluations of all deep value networks simulated under the ith branch in the specific situation.

In the deep value network integrated MCTS algorithm, a Monte Carlo search state tree is maintained. The iterative process of a single simulation is as follows:

Step 1: Get all the optional moves in the current situation;
Step 2: If the number of optional moves is 1, select the move as action and go to step 8;
Step 3: Query all branch states, that is, the state of the situation after the optional operation is performed, the number of times the corresponding state node is accessed in the Monte Carlo search state tree;
Step 4: If the state corresponds to the state node of the Monte Carlo search state tree is visited for the first time, ie $n'_i = 0$, the depth value network is used to evaluate the state of the situation and recursively updates the w'_i and n'_i state values of the state node of the simulation route;
Step 5: If the number of accesses to the status node of all branch states reaches the threshold L ($L = 10$ is set in this paper), go to step6; otherwise, go to step 7;
Step 6: Calculate the value of all alternative methods according to formula (4), sort these methods, select the highest valued action as the action method, and go to step 8;
Step 7: Select randomly one of the optional moves as the action method and go to step 8;

Step 8: Add the current position situations to the analogue path status set;
Step 9: Perform actions in the current situation and move to the next position situation;
Step 10: If the current situation corresponds to the number of rounds is <28, go to step1, otherwise enter step11;
Step 11: Use the Alpha-Beta algorithm to determine the outcome of the situation;
Step 12: Recursively update the w_i, n_i, w'_i, and n'_i state values of the corresponding state nodes in the Monte Carlo state tree corresponding to the state of the simulation path, and end the simulation.

Perform as many simulations as possible within a 20-second time frame, and then select the highest value according to the final state value and formula (4) of the corresponding state node in the Monte Carlo state tree for all the alternative branch situations in the current situation. Optional method is the best move. In the entire game, the same Monte Carlo status tree is maintained, making maximum use of historical search results.

4.2 Depth strategy network integrated MCTS algorithm

In-depth strategy network is specially used to provide decision-making reference opinions under various grid check situations, then the model can prune the game tree in the stage of legal decision-making and greatly reduce the size of the search space.

When using the in-depth strategy network to make a legitimate decision screening for the MCTS algorithm, first set the number of simulated pruning thresholds a, the number of branches threshold b and the number of full search rounds threshold c, this paper sets the threshold a is 2, the threshold b is 8 and c is 28. Three thresholds are considered when generating a set of legitimate decisions: If the current position is more than threshold a round away from the root of the game search tree, deep strategy network is not applied; otherwise, the total number of legal decisions is counted, and if the legal decision is not more than the threshold b, deep strategy network is not applied too; If the candidate legal strategy is more than the threshold b, the in-depth strategy network is used to score all legal branches under the current situation, and the best b moves are selected as the search alternative decision, and the expansion simulation is continued; The number of turns corresponding to the current situation is not less than the threshold c, and the Alpha-Beta search algorithm using rule compression is directly used to determine the outcome of the current simulation through complete search. In the MCTS algorithm of the deep strategy network integration, a Monte Carlo search state tree is maintained. The iterative process of a single simulation is as follows:

Step 1: If the current situation is the initial state of the search, that is, the root node of the Monte Carlo search state tree, record the number of rounds corresponding to the current situation, otherwise go to step2;
Step 2: Get all the optional moves in the current situation;
Step 3: If the current position is no more than threshold a round from the root of the search, and the number of available methods is more than threshold b, then use an in-depth strategy network for the current situation and select the b most according to the confidence level of the model output. Good practice as a new alternative;
Step 4: If the number of optional moves is 1, select the move as action and go to step9;
Step 5: Query all branch states, that is, the state of the situation after performing the optional operation, the number of times the corresponding state node is visited in the Monte Carlo state tree;
Step 6: If the number of visited nodes of all branch states reaches the threshold L ($L = 10$ is set in this paper), go to step 7; otherwise, go to step 8;
Step 7: Calculate the value of all alternative methods according to formula (4), sort these methods, select the one with the highest value as the action method, and go to step 9;

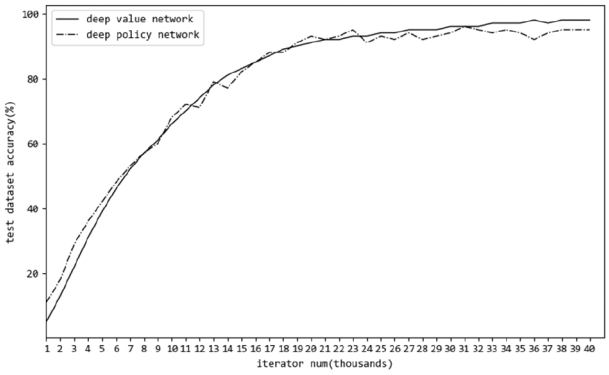

Fig. 7 *Two deep model training results*

Table 2 Game system round Robin results

	Value net + MCTS	Policy net + MCTS	Human
value net + MCTS	—	4:6	9:1
policy net + MCTS	5:5	—	10:0
human	2:8	1:9	—

Step 8: Select randomly one of the optional moves as the action method and go to step 9;
Step 9: Add the status of the current situation to the state collection of simulation channels;
Step 10: Perform actions in the current situation and move to the next position;
Step 11: If the current situation corresponds to the number of rounds is less than c, go to step 1, otherwise go to step12;
Step 12: Complete search using Alpha-Beta to determine the outcome of the situation;
Step 13: Recursively update the w_i and n_i state values of the corresponding state nodes in the Monte Carlo state tree corresponding to the state of the simulation path, and end the simulation.

Perform as many simulations as possible within a 20-second time frame, and then select the highest value according to the final state value and formula (4) of the corresponding state node in the Monte Carlo state tree for all the alternative branch situations in the current situation. Optional method is the best move. In the entire game, the same Monte Carlo status tree is maintained, making maximum use of historical search results.

5 Experiments and results

The experimental environment was: Ubuntu 16.04 × 64, Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40 GHz, GeForce GTX TITAN X, CUDA 8.0.44, cuDNN 5.1.5, TensorFlow [15] 1.0.0. This paper uses C++ to write a dots and boxes game system.

We first completed the supervised model training deep value network and deep strategy network model. The two model training results are shown in Fig. 7.

Second, MCTS and Alpha-Beta search algorithms are implemented, respectively. Then, the trained depth value network and depth strategy network are integrated with the MCTS algorithm and a complete search is started as soon as possible to truncate the MCTS simulation. Finally, the two integrated networks and the author (with the level of human common chess game) are held in a circular game. The results of the game are shown in Table 2. Among them, the value net + MCTS table, the MCTS algorithm representing deep value network integration, the policy net + MCTS represents the MCTS algorithm of the deep strategy network integration, human represents the author, the vertical axis player first, and the horizontal axis player.

As can be seen from Table 2, the MCTS algorithm integrated by depth value network is slightly stronger than the MCTS algorithm integrated by depth policy network. At the same time, the two

integrated algorithm game systems have reached a far higher level of intelligence than ordinary humans when solving dots and boxes chess game problems.

6 Conclusion

First, this paper analysed the characteristics of the rules of the dots and boxes game, and then proposes a matrix grid representation form of dots and boxes as the input of the neural network model. Then, two kinds of deep CNN models were designed, implemented and trained: deep value network for situation assessment and deep strategy network for recommendation. The model converged well on the data set. Correspondingly, this paper also proposes two integration models, one is the integration form of MCTS algorithm and deep value network model, and the other is the integration form of MCTS algorithm and in-depth strategy network model. Wherein, the depth value network is used to give a reference for the situation assessment, and a weighted sum is obtained with the Monte Carlo simulation of the situation assessment; the in-depth strategy network is used to score all legitimate moves and sort these moves to select the best n moves as the actual search branch, other movements have been abandoned. In both integrated models, Alpha-Beta complete search is used to truncate the Monte Carlo simulation process and improve simulation efficiency. The experiment confirmed that the two integrated models mentioned reached an impressive level of intelligence.

Since the CNN model has not yet been trained in all rounds of dots and boxes games, the model can be further improved after collecting richer match data. In addition, it is also possible to consider using the deep value network and the deep strategy network together in the same integration model, and cooperate with Alpha-Beta search to provide the game state tree pruning for the MCTS and further reduce the search space.

7 Acknowledgment

This work is supported by Key potential projects of Promoting Research Level program at Beijing Information Science and Technology University (NO. 5211910927), by Normal projects of General Science and Technology research program (NO. KM201911232002), by Science & Technology Innovation program for graduated students at Beijing Information Science and Technology University (NO. 5121911044), by Normal projects of promoting graduated education program at Beijing Information Science and Technology University (NO. 5121911019).

8 References

[1] Pearl, J.: 'The solution for the branching factor of the alpha-beta pruning algorithm and its optimality', *Commun. ACM*, 1982, **25**, (8), pp. 559–564
[2] Chaslot, G., Bakkes, S., Szita, I., *et al.*: 'Monte-Carlo tree search: a new framework for game AI'. Proc. of the Fourth Artificial Intelligence and Interactive Digital Entertainment Conf. (AIIDE), Palo Alto, CA, USA, 2008
[3] Mnih, V., Kavukcuoglu, K., Silver, D., *et al.*: 'Human-level control through deep reinforcement learning', *Nature*, 2015, **518**, (7540), p. 529
[4] Silver, D., Huang, A., Maddison, C.J., *et al.*: 'Mastering the game of Go with deep neural networks and tree search', *Nature*, 2016, **529**, (7587), pp. 484–489
[5] Silver, D., Schrittwieser, J., Simonyan, K., *et al.*: 'Mastering the game of Go without human knowledge', *Nature*, 2017, **550**, (7676), p. 354
[6] Qiu, X., Zhang, L., Ren, Y., *et al.*: 'Ensemble deep learning for regression and time series forecasting'. 2014 IEEE Symp. on Computational Intelligence in Ensemble Learning (CIEL), Orlando, FL, USA, 2014, pp. 1–6
[7] Lecun, Y., Bengio, Y.: 'Convolutional networks for images, speech, and time series', '*The handbook of brain theory and neural networks*' (MIT Press, Cambridge, MA, USA, 1998), pp. 1–14
[8] Moravčík, M., Schmid, M., Burch, N., *et al.*: 'Deepstack: expert-level artificial intelligence in no-limit poker[EB/OL]', arXiv preprint arXiv:1701.01724, 2017
[9] Berlekamp, E., Scott, K.: 'Forcing your opponent to stay in control of a loony dots-and-boxes endgame', '*More games of no chance*', vol. **42** (Berkeley, CA, 2000), pp. 317–330
[10] Berlekamp, E.R.: '*The dots and boxes game: sophisticated child's play*' (AK Peters/CRC Press, 2000)
[11] Clevert, D.A., Unterthiner, T., Hochreiter, S.: 'Fast and accurate deep network learning by exponential linear units (ELUs)', arXiv preprint arXiv:1511.07289, 2015
[12] Kingma, D.P., Ba, J.: 'Adam: a method for stochastic optimization', arXiv preprint arXiv:1412.6980, 2014
[13] Kocsis, L.: 'Bandit based Monte-Carlo planning'. European Conf. on Machine Learning, Berlin, Germany, 2006, pp. 282–293
[14] Garivier, A., Moulines, E.: 'On upper-confidence bound policies for switching bandit problems'. Int. Conf. on Algorithmic Learning Theory, Espoo, Finland, 2011, pp. 174–188
[15] https://tensorflow.google.cn/